

Bazy Danych

DOKUMENTACJA PROJEKTU Z BAZ DANYCH „HOTEL”

Oskar Gregorczyk

Gregorczyk Oskar

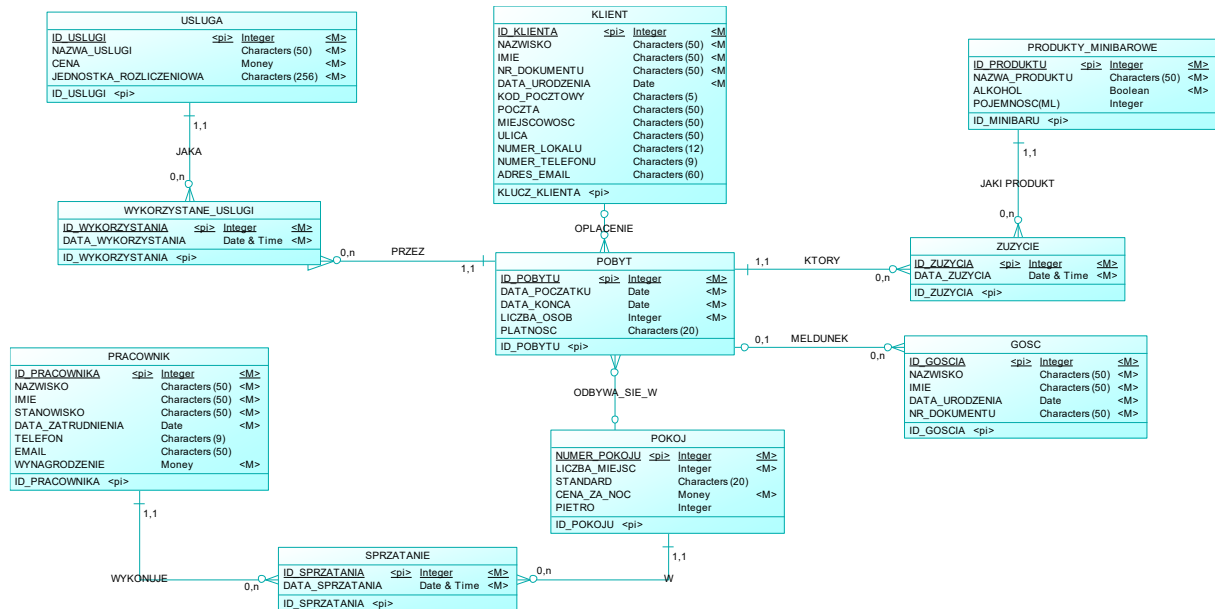
DIAGRAM KONCEPTUALNY

Diagram Konceptualny zawiera 10 encji:

- 1) **POBYT** – lista wszystkich zarezerwowanych pobyków w hotelu. Atrybutem jednoznacznie identyfikującym pobyt jest ID_POBYTU, różny dla każdej instancji. DATA_POCZATKU i DATA_KONCA to daty zameldowania i wymeldowania, dzięki nim możemy policzyć, ile dni trwa dana rezerwacja. LICZBA_OSOB to ilość osób, które przebywają w hotelu i są zameldowane na ten pobyt. PLATNOSC to dodatkowa i nieobowiązkowa informacja czy klient preferuje zapłacić kartą, gotówką czy może bonem, może być to też informacja o wpłaceniu zaliczki, lub braku pełnej opłaty.
- 2) **KLIENT**– lista osób, które zrobiły rezerwację w hotelu i ją opłaciły. Kluczem głównym jest sztuczny atrybut ID_KLIENTA. Encja zawiera NAZWISKO i IMIE, NR_DOKUMENTU, który został podany w trakcie składania rezerwacji i zameldowania się, DATA_URODZENIA, co pozwala określić pełnoletność, a dodatkowe atrybuty tj. KOD_POCZTOWY, POCZTA, MIEJSCOWOSC, ULICA, NUMER_LOKALU, NR_TELEFONU i ADRES_EMAIL służą do ułatwionej korespondencji z klientem.
- 3) **GOSC** – lista osób, które są zameldowane w hotelu. Klucz główny to ID_GOSCIA. Podobnie jak w przypadku klienta, encja zawiera NAZWISKO, IMIE, a także DATA_URODZENIA oraz NR_DOKUMENTU.
- 4) **PRODUKTY_MINIBAROWE** – każdy pokój zawiera lodówkę z minibarem, a to jest lista dostępnych produktów, uzupełnianych każdego dnia jednorazowo. Encja zawiera ID_PRODUKTU, NAZWA_PRODUKTU oraz koniecznie ALKOHOL, czy dany produkt zawiera alkohol czy nie. Dodatkową opcją jest POJEMNOSC.
- 5) **POKOJ** – jest to lista wszystkich pokoi hotelowych, które są dostępne dla gości hotelowych w celu noclegu. Klucz główny to NUMER_POKOJU, jest to

wystarczający atrybut, gdyż w hotelu nie ma dwóch pokoi o takim samym numerze. LICZBA_MIEJSC to liczba dostępnych w tym pokoju łóżek. STANDARD to słowny opis czy pokój jest klasy standardowej czy premium. CENA_ZA_NOC to ustalona indywidualnie cena, jaką musi zapłacić klient za nocleg w tym pokoju. PIETRO ułatwia klientom i pracownikom zlokalizowanie pokoju w ośrodku, jednakże nie jest to pole konieczne, ze względu, iż z reguły numery pokoju zawierają te informacje.

- 6) PRACOWNIK – lista wszystkich osób zatrudnionych w hotelu. Każdy pracownik ma indywidualny ID_PRACOWNIKA. Konieczne do zapisania w tabeli są NAZWISKO, IMIE, STANOWISKO, a także WYNAGRODZENIE. Dodatkowe atrybuty to TELEFON i EMAIL, w celu korespondencji z pracownikiem.
- 7) USLUGA – lista wszystkich dostępnych usług w hotelu. Oprócz indywidualnego ID_USLUGI, zawiera też NAZWA_USLUGI, CENA, a także JEDNOSTKA_ROZLICZENIOWA, gdyż niektóre usługi mogą być rozliczane jednorazowo tj. masaż lub jednorazowo.
- 8) WYKORZYSTANE_USLUGI – lista wszystkich wykorzystanych usług, zawiera ID_WYKORZYSTANIA, a także datę i godzinę wykorzystania danej usługi.
- 9) SPRZATANIE – lista wykonanych sprzątań, zawiera ID_SPRZATANIA, a także datę i godzinę posprzątania pokoju.
- 10) ZUZYCIE – lista zużytych produktów z minibarku, zawiera ID_ZUZYCIA oraz datę i godzinę wykorzystania produktu.

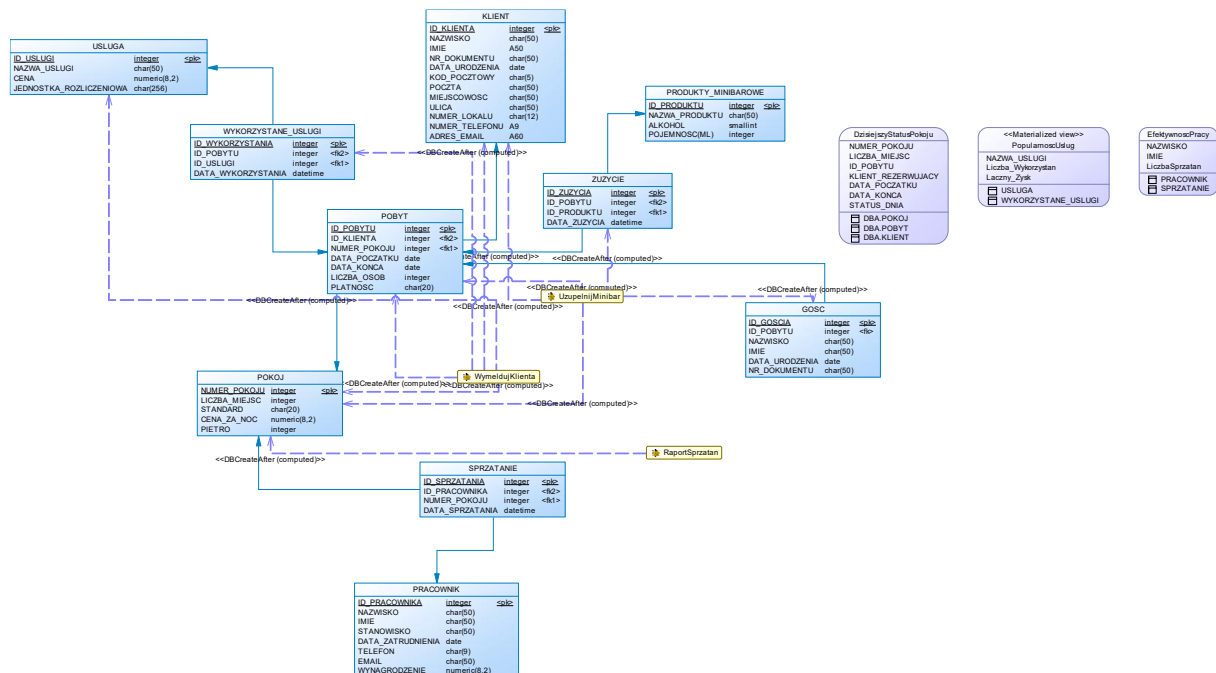
Relacje między encjami:

- 1) OPLACENIE KLIENT-POBYT (1 do N) – każdy pobyt jest opłacany przez jednego klienta, natomiast jeden klient może jednocześnie opłacać wiele pobytów
- 2) JAKI PRODUKT PRODUKTY_MINIBAROWE-ZUZYCIE (1 do N) – dany produkt może być zużyty wiele razy, ale w jednym rekordzie w tabeli ZUZYCIE musi pojawić się tylko jeden produkt.
- 3) KTÓRY POBYT-ZUZYCIE (1 do N) – dany pobyt może wykorzystać wiele produktów, a w jednym rekordzie w tabeli ZUZYCIE musi się pojawić tylko jedno ID_POBYTU, w trakcie którego zużyto produkt
- 4) MELDUNEK POBYT-GOSC (1 do N) – w ramach jednego pobytu może być zameldowanych kilku gości, natomiast jeden gość jest przypisany do jednego pobytu.
- 5) ODBYWA_SIE_W POKOJ-POBYT (1 do N) – jeden pobyt odbywa się w jednym pokoju, jednakże w jednym pokoju może być wiele pobytów.
- 6) WYKONUJE PRACOWNIK SPRZATANIE (1 do N) - jeden pracownik może posprzątać wiele pokoi, ale jedno sprzątnięcie wykonane jest przez tylko jednego pracownika.
- 7) W SPRZATANIE-POKOJ (1 do N) – jeden pokój może być posprzątany wiele razy, ale jedno sprzątnięcie dotyczy tylko jednego pokoju.

- 8) JAKA USŁUGA-POBYT (1 do N)- jedna usługa, może być wykorzystana przez wiele pobytów, ale jeden rekord w tabeli WYKORZYSTANE_USLUGI zawiera jedną usługę.
- 9) PRZEZ POBYT-WYKORZYSTANE_USLUGI (1 do N)- jeden pobyt może wykorzystać wiele usług, ale jeden rekord w tabeli WYKORZYSTANE_USLUGI zawiera jeden pokój.

DIAGRAM FIZYCZNY

W diagramie fizycznej do każdej relacji 1 do N, do tabeli, która jest przy N dodajemy klucz obcy, czyli klucz główny drugiej tabeli.



W ten sposób uzyskujemy table:

- 1) KLIENT (bez zmian)
- 2) POBYT – zawiera klucz obcy KLIENT, czyli ID_KLIENTA, a także NUMER_POKOJU, czyli klucz główny tabeli POKOJ,
- 3) GOSC – zawiera klucz obcy POBYT, czyli ID_POBYTU,
- 4) PRODUKTY MINIBAROWE (bez zmian)
- 5) ZUZYCIE- ID_PRODUKTU (klucz główny PRODUKTY_MINIBAROWE) i ID_POBYTU (klucz główny POBYT),
- 6) POKOJ (bez zmian)
- 7) PRACOWNIK (bez zmian)

- 8) SPRZATANIE –tabela zawiera NUMER_POKOJU i ID_PRACOWNIKA jako klucze obce,
- 9) USLUGA (bez zmian)
- 10) WYKORZYSTANE_USLUGI- zawiera klucz obcy ID_USLUGI (tabela USLUGA), a także klucz obcy ID_POBYTU (POBYT),

WIDOKI

1. DzisiejszyStatusPokoju

SQL ANYWHERE

```
ALTER VIEW "DBA"."DzisiejszyStatusPokoju" AS
SELECT p.NUMER_POKOJU, p.LICZBA_MIEJSC, pb.ID_POBYTU,
       k.IMIE || ' ' || k.NAZWISKO AS KLIENT_REZERWUJACY, pb.DATA_POCZATKU,
       pb.DATA_KONCA,
       CASE
         WHEN pb.DATA_POCZATKU= CURRENT DATE THEN 'Przyjazd dziś'
         WHEN pb.DATA_KONCA= CURRENT DATE THEN 'Wyjazd dziś'
         WHEN CURRENT DATE BETWEEN pb.DATA_POCZATKU AND pb.DATA_KONCA THEN
           'W trakcie'
         ELSE 'Wolny'
       END AS STATUS_DNIA
FROM
  DBA.POKOJ p LEFT JOIN  DBA.POBYT pb
ON p.NUMER_POKOJU = pb.NUMER_POKOJU
  AND CURRENT DATE BETWEEN pb.DATA_POCZATKU AND pb.DATA_KONCA
LEFT JOIN
  DBA.KLIENT k ON pb.ID_KLIENTA = k.ID_KLIENTA
```

MYSQL

```
CREATE VIEW DzisiejszyStatusPokoju AS
SELECT
  p.NUMER_POKOJU,
  p.LICZBA_MIEJSC,
  pb.ID_POBYTU,
  CONCAT(k.IMIE, ' ', k.NAZWISKO) AS KLIENT_REZERWUJACY,
  pb.DATA_POCZATKU,
  pb.DATA_KONCA,
  CASE
    WHEN pb.DATA_POCZATKU = CURDATE() THEN 'Przyjazd dziś'
    WHEN pb.DATA_KONCA = CURDATE() THEN 'Wyjazd dziś'
    WHEN CURDATE() BETWEEN pb.DATA_POCZATKU AND pb.DATA_KONCA THEN 'W
trakcie'
    ELSE 'Wolny'
  END AS STATUS_DNIA
FROM
  POKOJ p
LEFT JOIN POBYT pb ON p.NUMER_POKOJU = pb.NUMER_POKOJU
  AND CURDATE() BETWEEN pb.DATA_POCZATKU AND pb.DATA_KONCA
LEFT JOIN
  KLIENT k ON pb.ID_KLIENTA = k.ID_KLIENTA;
```

Ten widok to przydatne narzędzie dla Recepcjonisty, który wpisując rezerwację w łatwy sposób może sprawdzić które pokoje są w danym dniu dostępne. Kolumnami są NUMER_POKOJU, LICZBA_MIEJSC, a następnie, jeśli konkretny pokój aktualnie jest zarezerwowany, imię i nazwisko KLIENT_REZERWUJĄCY, a także DATA_POCZATKU i DATA_KONCA tej rezerwacji. Następnie zależnie czy dziś jest DATA_POCZATKU konkretnej rezerwacji widzimy komunikat „Przyjazd dziś”, DATA_KONCA „Wyjazd dziś” lub dzisiejsza data jest pomiędzy nimi to komunikat brzmi „W trakcie”, jeśli dany pokój jednak nie jest zarezerwowany to widzimy napis „Wolny”. Tabela POKOJ jest połączona z tabelą POBYT za pomocą LEFT JOIN, dzięki temu widzimy wszystkie pokoje, niezależnie czy jest w nich jakaś rezerwacja, a także

2. EfektywnoscPracy

SQL ANYWHERE

```
ALTER VIEW "DBA"."EfektywnoscPracy" AS
```

```
SELECT PRACOWNIK.NAZWISKO, PRACOWNIK.IMIE,  
COUNT(SPRZATANIE.NUMER_POKOJU)AS LiczbaSprzatan  
FROM PRACOWNIK, SPRZATANIE  
WHERE PRACOWNIK.ID_PRACOWNIKA=SPRZATANIE.ID_PRACOWNIKA  
GROUP BY PRACOWNIK.NAZWISKO, PRACOWNIK.IMIE  
ORDER BY LiczbaSprzatan DESC;
```

MYSQL

```
CREATE VIEW EfektywnoscPracy AS  
SELECT  
    p.NAZWISKO,  
    p.IMIE,  
    COUNT(s.NUMER_POKOJU) AS LiczbaSprzatan  
FROM  
    PRACOWNIK p  
JOIN  
    SPRZATANIE s ON p.ID_PRACOWNIKA = s.ID_PRACOWNIKA  
GROUP BY  
    p.NAZWISKO,  
    p.IMIE  
ORDER BY
```

LiczbaSprzatan DESC;

Widok przedstawia, NAZWISKO i IMIE danego pracownika, a także liczbę posprzątaných przez niego pokoiów. Dzięki temu widokowi kierownik jest w stanie zweryfikować efektywność pracy i dać premie wyróżniającym się osobom. Lista osób jest posortowana malejąco, więc osoba, która posprzątała najwięcej pokoiów jest na górze.

3. PopularnoscUslug

```
CREATE MATERIALIZED VIEW "DBA"."PopularnoscUslug"
```

```
IN "system" AS
```

```
SELECT
```

```
    USLUGA.NAZWA_USLUGI,
```

```
    COUNT(WYKORZYSTANE_USLUGI.ID_USLUGI) AS Liczba_Wykorzystan,
```

```
    SUM(USLUGA.CENA) AS Laczny_Zysk
```

```
FROM USLUGA
```

```
LEFT JOIN WYKORZYSTANE_USLUGI ON USLUGA.ID_USLUGI =  
WYKORZYSTANE_USLUGI.ID_USLUGI
```

```
GROUP BY USLUGA.NAZWA_USLUGI
```

Ten widok, przedstawia wszystkie usługi, a także ile razy zostały wykorzystane przez klientów, oprócz tego kwota łącznie zarobionych pieniędzy dzięki danej usłudze.

FUNKCJE WBUDOWANE

1. StanPokojuWDniu

SQL ANYWHERE

```
ALTER FUNCTION "DBA"."StanPokojuWDniu"( IN nNUMER_POKOJU INT, dDATA DATE)
```

```
RETURNS INT
```

```
DETERMINISTIC
```

```
BEGIN
```

```
    DECLARE _czy_zajety INT;
```

```
    SET _czy_zajety=(
```

```
    SELECT COUNT(*)
```

```
    FROM POBYT
```

```
    WHERE NUMER_POKOJU=nNUMER_POKOJU
```

```
AND dDATA BETWEEN DATA_POCZATKU AND DATA_KONCA
);
IF _czy_zajety>0 THEN RETURN 0;
ELSE
    RETURN 1;
ENDIF;
END
```

MYSQL

```
DELIMITER $$
```

```
CREATE FUNCTION StanPokojuWDniu(nNUMER_POKOJU INT, dDATA DATE)
RETURNS INT
DETERMINISTIC
BEGIN
    DECLARE czy_zajety INT;
    SELECT COUNT(*) INTO czy_zajety
    FROM POBYT
    WHERE NUMER_POKOJU = nNUMER_POKOJU
    AND dDATA BETWEEN DATA_POCZATKU AND DATA_KONCA;
    IF czy_zajety > 0 THEN
        RETURN 0;
    ELSE
        RETURN 1;
    END IF;
END $$
```

```
DELIMITER ;
```

Ta funkcja zwraca, czy dany Pokój w danym terminie jest zajęty czy też nie. Parametry funkcji, to NUMER_POKOJU jaki chcemy sprawdzić, a także data, która nas interesuje. Jeśli pokój jest zajęty, funkcja zwraca 0 (nie można rezerwować pobytu w tym pokoju), natomiast, gdy pokój jest wolny wartość _czy_zajety jest równa 1 (ten pokój można zarezerwować).

2. Kolor Opaski

SQL ANYWHERE

```
ALTER FUNCTION "DBA"."KolorOpaski"( iID_GOSCIA INT )
RETURNS VARCHAR(20)
DETERMINISTIC
BEGIN
    DECLARE _wiek_goscia INT;
```



```
DECLARE _status_goscia VARCHAR(20);
SET _wiek_goscia=(
    SELECT datediff(year, GOSC.DATA_URODZENIA, CURRENT DATE)
    FROM GOSC
    WHERE GOSC.ID_GOSCIA=iID_GOSCIA
);
IF _wiek_goscia>=18 THEN
    SET _status_goscia = 'Dorosly';
ELSE
    SET _status_goscia ='Dziecko';
END IF;
RETURN _status_goscia;
END

DELIMITER $$
```

MYSQL

```
CREATE FUNCTION KolorOpaski(iID_GOSCIA INT)
RETURNS VARCHAR(20)
DETERMINISTIC
BEGIN
    DECLARE wiek_goscia INT;
    DECLARE status_goscia VARCHAR(20);

    SELECT TIMESTAMPDIFF(YEAR, DATA_URODZENIA, CURDATE()) INTO wiek_goscia
    FROM GOSC
    WHERE ID_GOSCIA = iID_GOSCIA;

    IF wiek_goscia >= 18 THEN
        SET status_goscia = 'Dorosly';
    ELSE
        SET status_goscia = 'Dziecko';
    END IF;

    RETURN status_goscia;
END $$

DELIMITER ;
```

Przy zameldowaniu gości, recepcjonista automatycznie sprawdza pełnoletniość, co pomaga przy odpowiednim doborze koloru opasek. Bardzo często w hotelach typu All-Inclusive dorośli oraz dzieci mają różne kolory opasek, co pomaga np. Barmanom lub kelnerom, aby w łatwy sposób dowiedzieć się, czy dana osoba może pić napoje zawierające alkohol. Funkcja oblicza wiek gościa, a następnie zwraca w sposób jednoznaczny wynik „Dorośli” lub „Dziecko”. Jako argument przyjmuje ID_GOSCIA.

3. SprzątaniePokoju

SQL ANYWHERE

```
ALTER FUNCTION "DBA"."SprzątaniePokoju"( iID_POKOJU INT )
```

```
RETURNS VARCHAR(50)
```

```
DETERMINISTIC
```

```
BEGIN
```

```
    DECLARE "StanPokoju" VARCHAR(50);
```

```
    DECLARE _czy_dzis_sprzatany INT;
```

```
    DECLARE _typ_dnia_rezerwacji VARCHAR(20);
```

```
    SET _czy_dzis_sprzatany=(
```

```
    SELECT count(*)
```

```
    FROM SPRZATANIE
```

```
    WHERE NUMER_POKOJU = iID_POKOJU
```

```
    AND CAST(DATA_SPRZATANIA AS DATE) = CURRENT DATE
```

```
    );
```

```
    IF _czy_dzis_sprzatany > 0 THEN
```

```
        RETURN 'Posprzatany'
```

```
    END IF;
```

```
    SET _typ_dnia_rezerwacji=(
```

```
    SELECT
```

```
        CASE
```

```
            WHEN POBYT.DATA_POCZATKU = CURRENT DATE THEN 'PRZYJAZD'
```

```
            WHEN POBYT.DATA_KONCA = CURRENT DATE THEN 'WYJAZD'
```

```
            ELSE 'W_TRAKCIE'
```

```
        END
```

```
    FROM POBYT
```

```
    WHERE NUMER_POKOJU = iID_POKOJU
```

```
    AND CURRENT DATE BETWEEN POBYT.DATA_POCZATKU AND
```

```
    POBYT.DATA_KONCA
```

```
    );
```

```
IF _typ_dnia_rezerwacji = 'PRZYJAZD' THEN
    RETURN 'Klient przyjeżdża';
ELSEIF _typ_dnia_rezerwacji = 'WYJAZD' THEN
    RETURN 'Klient wyjeżdża';
ELSEIF _typ_dnia_rezerwacji = 'W_TRAKCIE' THEN
    RETURN 'Posprzataj';
ELSE
    RETURN 'Nie wymaga sprzątania';
END IF;
END
```

MYSQL

```
DELIMITER $$
```

```
CREATE FUNCTION SprzataniePokoju(iID_POKOJU INT)
RETURNS VARCHAR(50)
DETERMINISTIC
BEGIN
    DECLARE czy_dzis_sprzatany INT;
    DECLARE typ_dnia_rezerwacji VARCHAR(20);

    SELECT COUNT(*) INTO czy_dzis_sprzatany
    FROM SPRZATANIE
    WHERE NUMER_POKOJU = iID_POKOJU
    AND DATE(DATA_SPRZATANIA) = CURDATE();

    IF czy_dzis_sprzatany > 0 THEN
        RETURN 'Posprzatany';
    END IF;

    SELECT
        CASE
            WHEN DATA_POCZATKU = CURDATE() THEN 'PRZYJAZD'
            WHEN DATA_KONCA = CURDATE() THEN 'WYJAZD'
            ELSE 'W_TRAKCIE'
        END INTO typ_dnia_rezerwacji
    FROM POBYT
    WHERE NUMER_POKOJU = iID_POKOJU
    AND CURDATE() BETWEEN DATA_POCZATKU AND DATA_KONCA;
```

```
IF typ_dnia_rezerwacji = 'PRZYJAZD' THEN
    RETURN 'Klient przyjeżdża';
ELSEIF typ_dnia_rezerwacji = 'WYJAZD' THEN
    RETURN 'Klient wyjeżdża';
ELSEIF typ_dnia_rezerwacji = 'W_TRAKCIE' THEN
    RETURN 'Posprzątaj';
ELSE
    RETURN 'Nie wymaga sprzątania';
END IF;
END $$
```

DELIMITER ;

Ta funkcja przydaje się Kierownikowi, odpowiedzialnemu za rozdysponowanie pracy sprzątaczym. Funkcja zwraca 4 różne wartości:

- „Posprzątany” – jeśli pokój został już posprzątany dzisiaj, czyli w tabeli SPRZATANIE, znajduje się rekord o danym NUMER_POKOJU oraz DATA_SPRZATANIA jest taka jak data danego nie

- „Klient przyjeżdża” – jeśli w danym pokoju, dziś zaczyna się rezerwacja, pokój musi być posprzątany przed godziną zameldowania.

- „Klient wyjeżdża”- jeśli w danym pokoju, dziś kończy się rezerwacja, pokój ma być posprzątany po wymeldowaniu gości.

- „Posprzątaj”- jeśli obecnie w danym pokoju jest rezerwacja i pokój nie był jeszcze posprzątany danego dnia

- „Nie wymaga sprzątania”- gdy w danym pokoju, nie ma rezerwacji, ani żaden klient nie przyjeżdża w tym dniu.

Jako argument przyjmuje NUMER_POKOJU.

PROCEDURY

1. WymeldujKlienta

SQL ANYWHERE

```
ALTER PROCEDURE "DBA"."WymeldujKlienta"(IN pPOBYT INT)
BEGIN
    DECLARE _suma_nocleg NUMERIC(10,2) = 0;
    DECLARE _suma_uslugi NUMERIC(10,2) = 0;
    DECLARE _suma_calkowita NUMERIC(10,2) = 0;

    SET _suma_nocleg =(
    SELECT
```

```
CASE
    WHEN DATEDIFF(day, POBYT.DATA_POCZATKU, POBYT.DATA_KONCA) = 0
THEN POKOJ.CENA_ZA_NOC
    ELSE DATEDIFF(day, POBYT.DATA_POCZATKU, POBYT.DATA_KONCA) *
POKOJ.CENA_ZA_NOC
END
FROM POBYT
JOIN POKOJ ON POBYT.NUMER_POKOJU = POKOJ.NUMER_POKOJU
WHERE POBYT.ID_POBYTU = pPOBYT
);

SET _suma_uslugi =(
    SELECT SUM(USLUGA.CENA)
    FROM WYKORZYSTANE_USLUGI
    JOIN USLUGA ON
WYKORZYSTANE_USLUGI.ID_USLUGI=DBA.USLUGA.ID_USLUGI
    WHERE WYKORZYSTANE_USLUGI.ID_POBYTU=pPOBYT
);

SET _suma_calkowita=_suma_nocleg+ISNULL(_suma_uslugi, 0);

SELECT POBYT.ID_POBYTU, KLIENT.NAZWISKO, KLIENT.IMIE,
POBYT.NUMER_POKOJU,
    _suma_nocleg, _suma_uslugi, _suma_calkowita, POBYT.PLATNOSC
FROM POBYT JOIN KLIENT ON POBYT.ID_KLIENTA=KLIENT.ID_KLIENTA
WHERE POBYT.ID_POBYTU=pPOBYT;
END
```

MYSQL

```
DELIMITER $$
```

```
CREATE PROCEDURE WymeldujKlienta(IN pPOBYT INT)
BEGIN
    DECLARE suma_nocleg DECIMAL(10,2);
    DECLARE suma_uslugi DECIMAL(10,2);
    DECLARE suma_calkowita DECIMAL(10,2);
```

```
SELECT
    CASE
```

```
        WHEN DATEDIFF(pb.DATA_KONCA, pb.DATA_POEZATKU) = 0 THEN
pk.CENA_ZA_NOC
        ELSE DATEDIFF(pb.DATA_KONCA, pb.DATA_POEZATKU) * pk.CENA_ZA_NOC
    END INTO suma_nocleg
FROM POBYT pb
JOIN POKOJ pk ON pb.NUMER_POKOJU = pk.NUMER_POKOJU
WHERE pb.ID_POBYTU = pPOBYT;

SELECT
    COALESCE(SUM(u.CENA), 0) INTO suma_uslugi
FROM WYKORZYSTANE_USLUGI wu
JOIN USLUGA u ON wu.ID_USLUGI = u.ID_USLUGI
WHERE wu.ID_POBYTU = pPOBYT;

SET suma_calkowita = suma_nocleg + suma_uslugi;

SELECT
    pb.ID_POBYTU,
    k.NAZWISKO,
    k.IMIE,
    pb.NUMER_POKOJU,
    suma_nocleg AS Koszt_Noclegu,
    suma_uslugi AS Koszt_Uslug,
    suma_calkowita AS Do_Zaplaty,
    pb.PLATNOSC
FROM POBYT pb
JOIN KLIENT k ON pb.ID_KLIENTA = k.ID_KLIENTA
WHERE pb.ID_POBYTU = pPOBYT;
END $$
```

DELIMITER ;

Ta procedura przydatna jest, gdy pobyt kończy się i chcemy rozliczyć się z klientem za jego pobyt. Argumentem procedury jest ID_POBYTU, który chcemy wymeldować. Cena za cały pobyt składa się z ceny za sam nocleg, a także z sumy cen wykorzystanych usług w trakcie tego pobytu. Zmienna _suma_nocleg jest to iloczyn liczby dni, jak długo trwał pobyt, a także ceny za jedną noc w danym pokoju, natomiast _suma_uslugi, to suma cen usług, z jakich skorzystał pobyt. Procedura na końcu wyświetla, ID_POBYTU, NAZWISKO, IMIE, NUMER_POKOJU, a także kwotę za nocleg, za usługi, i łączną kwotę do zapłaty.

2. UzupełnijMinibar – lokalna transakcja

SQL ANYWHERE

```
ALTER PROCEDURE "DBA"."UzupelnijMinibar"(
    IN iID_POBYTU INT,
    IN iP1 INT DEFAULT NULL, IN iP2 INT DEFAULT NULL,
    IN iP3 INT DEFAULT NULL, IN iP4 INT DEFAULT NULL,
    IN iP5 INT DEFAULT NULL, IN iP6 INT DEFAULT NULL,
    IN iP7 INT DEFAULT NULL
)
BEGIN
    IF iP1 IS NOT NULL THEN INSERT INTO "DBA"."ZUZYCIE" (ID_PRODUKTU,
ID_POBYTU, DATA_ZUZYCIA) VALUES (iP1, iID_POBYTU, CURRENT_TIMESTAMP); END
IF;
    IF iP2 IS NOT NULL THEN INSERT INTO "DBA"."ZUZYCIE" (ID_PRODUKTU,
ID_POBYTU, DATA_ZUZYCIA) VALUES (iP2, iID_POBYTU, CURRENT_TIMESTAMP); END
IF;
    IF iP3 IS NOT NULL THEN INSERT INTO "DBA"."ZUZYCIE" (ID_PRODUKTU,
ID_POBYTU, DATA_ZUZYCIA) VALUES (iP3, iID_POBYTU, CURRENT_TIMESTAMP); END
IF;
    IF iP4 IS NOT NULL THEN INSERT INTO "DBA"."ZUZYCIE" (ID_PRODUKTU,
ID_POBYTU, DATA_ZUZYCIA) VALUES (iP4, iID_POBYTU, CURRENT_TIMESTAMP); END
IF;
    IF iP5 IS NOT NULL THEN INSERT INTO "DBA"."ZUZYCIE" (ID_PRODUKTU,
ID_POBYTU, DATA_ZUZYCIA) VALUES (iP5, iID_POBYTU, CURRENT_TIMESTAMP); END
IF;
    IF iP6 IS NOT NULL THEN INSERT INTO "DBA"."ZUZYCIE" (ID_PRODUKTU,
ID_POBYTU, DATA_ZUZYCIA) VALUES (iP6, iID_POBYTU, CURRENT_TIMESTAMP); END
IF;
    IF iP7 IS NOT NULL THEN INSERT INTO "DBA"."ZUZYCIE" (ID_PRODUKTU,
ID_POBYTU, DATA_ZUZYCIA) VALUES (iP7, iID_POBYTU, CURRENT_TIMESTAMP); END
IF;
    COMMIT;
    MESSAGE 'Produkty zostały pomyślnie dodane do tabeli ZUZYCIE.' TO CLIENT;

EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        MESSAGE 'BŁĄD: Nie udało się dodać produktów. Transakcja wycofana.' TO
CLIENT;
END
```

MYSQL

DELIMITER \$\$

```
CREATE PROCEDURE UzupełnijMinibar(
    IN iID_POBYTU INT,
    IN iP1 INT, IN iP2 INT,
    IN iP3 INT, IN iP4 INT,
    IN iP5 INT, IN iP6 INT,
    IN iP7 INT
)
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'BŁĄD: Nie udało się dodać
produktów. Transakcja wycofana.';
    END;

    START TRANSACTION;

    IF iP1 IS NOT NULL THEN INSERT INTO ZUZYCIE (ID_PRODUKTU, ID_POBYTU,
DATA_ZUZYCIA) VALUES (iP1, iID_POBYTU, NOW()); END IF;
    IF iP2 IS NOT NULL THEN INSERT INTO ZUZYCIE (ID_PRODUKTU, ID_POBYTU,
DATA_ZUZYCIA) VALUES (iP2, iID_POBYTU, NOW()); END IF;
    IF iP3 IS NOT NULL THEN INSERT INTO ZUZYCIE (ID_PRODUKTU, ID_POBYTU,
DATA_ZUZYCIA) VALUES (iP3, iID_POBYTU, NOW()); END IF;
    IF iP4 IS NOT NULL THEN INSERT INTO ZUZYCIE (ID_PRODUKTU, ID_POBYTU,
DATA_ZUZYCIA) VALUES (iP4, iID_POBYTU, NOW()); END IF;
    IF iP5 IS NOT NULL THEN INSERT INTO ZUZYCIE (ID_PRODUKTU, ID_POBYTU,
DATA_ZUZYCIA) VALUES (iP5, iID_POBYTU, NOW()); END IF;
    IF iP6 IS NOT NULL THEN INSERT INTO ZUZYCIE (ID_PRODUKTU, ID_POBYTU,
DATA_ZUZYCIA) VALUES (iP6, iID_POBYTU, NOW()); END IF;
    IF iP7 IS NOT NULL THEN INSERT INTO ZUZYCIE (ID_PRODUKTU, iID_POBYTU,
DATA_ZUZYCIA) VALUES (iP7, iID_POBYTU, NOW()); END IF;

    COMMIT;

    SELECT 'Produkty zostały pomyślnie dodane do tabeli ZUZYCIE.' AS Status;

END $$
```


DELIMITER ;

Ta procedura jest to lokalna transakcja dodająca kilka wierszy do tabeli ZUZYCIE, jako argumenty przyjmuje ID_POBYTU, dla którego został uzupełniony minibar, a także do siedmiu ID_PRODUKTU, ponieważ tyle maksymalnie produktów jest w tabeli PRODUKTY_MINIBAROWE. W zależności, ile produktów zostało podanych jako argument, tyle wierszy zostanie dodanych do tabeli ZUZYCIE z aktualną godziną wpisania.

3. Raport Sprzatan

SQL ANYWHERE

```
ALTER PROCEDURE "DBA"."RaportSprzatan"()
RESULT( NrPokoju INT, CzyDoSprzatania VARCHAR(30) )
BEGIN
    DECLARE _nr_pokoju VARCHAR(10);
    DECLARE _status_sprzatania VARCHAR(30);

    DECLARE kursor_pokoi DYNAMIC SCROLL CURSOR FOR
        SELECT NUMER_POKOJU FROM POKOJ;

    OPEN kursor_pokoi;

    petla_raportu: LOOP
        FETCH NEXT kursor_pokoi INTO _nr_pokoju;
        IF SQLCODE <> 0 THEN LEAVE petla_raportu; END IF;
        SET _status_sprzatania = "DBA"."SprzataniePokoju"(_nr_pokoju);
        MESSAGE 'Pokoj: '||_nr_pokoju||' '||_status_sprzatania TO CLIENT;
    END LOOP petla_raportu;

    CLOSE kursor_pokoi;
    DEALLOCATE kursor_pokoi;
END
```

MYSQL

```
CREATE PROCEDURE RaportSprzatan()
BEGIN
    DECLARE done INT DEFAULT FALSE;
    DECLARE nr_pokoju_var INT;
    DECLARE status_sprzatania_var VARCHAR(50);
```

```
DECLARE kursor_pokoi CURSOR FOR SELECT NUMER_POKOJU FROM POKOJ;  
DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;  
  
CREATE TEMPORARY TABLE IF NOT EXISTS WynikiRaportu (  
    Log VARCHAR(255)  
);  
  
TRUNCATE TABLE WynikiRaportu;  
  
OPEN kursor_pokoi;  
  
petla_raportu: LOOP  
    FETCH kursor_pokoi INTO nr_pokoju_var;  
  
    IF done THEN  
        LEAVE petla_raportu;  
    END IF;  
  
    SET status_sprzatania_var = SprzataniePokoju(nr_pokoju_var);  
  
    INSERT INTO WynikiRaportu (Log)  
    VALUES (CONCAT('Pokoje: ', nr_pokoju_var, ' Status: ', status_sprzatania_var));  
  
END LOOP petla_raportu;  
  
CLOSE kursor_pokoi;  
  
SELECT Log FROM WynikiRaportu;  
  
DROP TEMPORARY TABLE WynikiRaportu;  
  
END $$
```

DELIMITER ;

Ta procedura wykorzystuje kursor, który pobiera NUMER_POKOJU z tabeli POKOJ, następnie w pętli petla_raportu, wypisuje do klienta wiadomość, z NUMER_POKOJU, a także wynikiem funkcji opisanej wyżej SprzataniePokoju(). W ten sposób Kierownik może w łatwy sposób zobaczyć całą listę pokoi i które pokoje wymagają sprzątnia, a które są już posprzątane.

WYZWALACZE

1. tr_automatycznie_dodaj_do_gosc

SQL ANYWHERE

```
ALTER TRIGGER "tr_automatycznie_dodaj_do_gosc" AFTER INSERT, UPDATE
ORDER 2 ON "DBA"."POBYT"
REFERENCING NEW AS new_pobyt
FOR EACH ROW
BEGIN
    INSERT INTO "DBA"."GOSC" (ID_POBYTU, IMIE, NAZWISKO, DATA_URODZENIA,
NR_DOKUMENTU)
    SELECT
        new_pobyt.ID_POBYTU,
        KLIENT.IMIE,
        KLIENT.NAZWISKO,
        KLIENT.DATA_URODZENIA,
        KLIENT.NR_DOKUMENTU
    FROM "DBA"."KLIENT"
    WHERE KLIENT.ID_KLIENTA = new_pobyt.ID_KLIENTA;
END
```

MYSQL

DELIMITER \$\$

```
CREATE TRIGGER tr_automatycznie_dodaj_do_gosc_INSERT
AFTER INSERT ON POBYT
FOR EACH ROW
BEGIN
    INSERT INTO GOSC (ID_POBYTU, IMIE, NAZWISKO, DATA_URODZENIA,
NR_DOKUMENTU)
    SELECT
        NEW.ID_POBYTU,
        k.IMIE,
        k.NAZWISKO,
        k.DATA_URODZENIA,
        k.NR_DOKUMENTU
    FROM KLIENT k
    WHERE k.ID_KLIENTA = NEW.ID_KLIENTA;
END $$
```

DELIMITER ;

Oskar Gregorczyk

DELIMITER \$\$

```
CREATE TRIGGER tr_automatycznie_dodaj_do_gosc_UPDATE
AFTER UPDATE ON POBYT
FOR EACH ROW
BEGIN
    INSERT INTO GOSC (ID_POBYTU, IMIE, NAZWISKO, DATA_URODZENIA,
NR_DOKUMENTU)
    SELECT
        NEW.ID_POBYTU,
        k.IMIE,
        k.NAZWISKO,
        k.DATA_URODZENIA,
        k.NR_DOKUMENTU
    FROM KLIENT k
    WHERE k.ID_KLIENTA = NEW.ID_KLIENTA;
END $$
```

DELIMITER ;

Recepcjonista uzupełniający bazę danych najpierw wpisuje nowego klienta, lub znajduje jego ID, jeśli miał już rezerwację w przeszłości, a dopiero potem uzupełnia tabelę POBYT, gdyż musi znaleźć się tam klucz obcy ID_KLIENTA. Po dodaniu nowego rekordu w tabeli POBYT, do tabeli GOSC automatycznie dodawany jest klient jako jeden z gości danego pobytu.

2. tr_blokada_podwójnej_rezerwacji

SQL ANYWHERE

```
ALTER TRIGGER "tr_blokada_podwójnej_rezerwacji" BEFORE INSERT, UPDATE
ORDER 1 ON "DBA"."POBYT"
REFERENCING NEW AS new_name
FOR EACH ROW
BEGIN
    IF EXISTS (
        SELECT 1
        FROM POBYT
        WHERE NUMER_POKOJU = new_name.NUMER_POKOJU
        AND ID_POBYTU <> new_name.ID_POBYTU
        AND new_name.DATA_POCZATKU < DATA_KONCA
        AND new_name.DATA_KONCA > DATA_POCZATKU
    )
)
```

Oskar Gregorczyk

```
THEN
    RAISERROR 20001 'Ten pokoj jest juz zarezerwowany w tym terminie';
END IF;
```

```
END
```

MYSQL

```
DELIMITER $$
```

```
CREATE TRIGGER tr_blokada_podwojnej_rezerwacji_INSERT
BEFORE INSERT ON POBYT
FOR EACH ROW
BEGIN
    IF EXISTS (
        SELECT 1
        FROM POBYT
        WHERE NUMER_POKOJU = NEW.NUMER_POKOJU
        -- Warunek nachodzenia na siebie dat:
        AND NEW.DATA_POCZATKU < DATA_KONCA
        AND NEW.DATA_KONCA > DATA_POCZATKU
    )
    THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Ten pokoj jest juz zarezerwowany w tym terminie';
    END IF;
END $$
```

```
DELIMITER ;
DELIMITER $$
```

```
CREATE TRIGGER tr_blokada_podwojnej_rezerwacji_UPDATE
BEFORE UPDATE ON POBYT
FOR EACH ROW
BEGIN
    IF EXISTS (
        SELECT 1
        FROM POBYT
        WHERE NUMER_POKOJU = NEW.NUMER_POKOJU
        AND ID_POBYTU <> NEW.ID_POBYTU
        AND NEW.DATA_POCZATKU < DATA_KONCA
        AND NEW.DATA_KONCA > DATA_POCZATKU
    )
    )
```

```
THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Ten pokój jest już zarezerwowany w tym terminie';
END IF;
END $$
```

DELIMITER ;

W tym samym czasie, nie jest możliwe, aby w jednym pokoju miały rezerwację dwa różne pobyty. Dlatego ten wyzwalacz, po próbie dodania nowego rekordu, sprawdza czy daty nie pokrywają się z zrobionymi już rezerwacjami, jeśli daty kolidują użytkownik widzi komunikat i nie jest w stanie dodać takiego wiersza.

3. tr_dodawanie_do_gosc

```
ALTER TRIGGER "tr_dodawanie_do_gosc" BEFORE INSERT, UPDATE
ORDER 1 ON "DBA"."GOSC"
REFERENCING OLD AS old_name NEW AS new_name
FOR EACH ROW
BEGIN
    IF EXISTS (
        SELECT GOSC.ID_POBYTU, COUNT(GOSC.ID_GOSCIA), POKOJ.LICZBA_MIEJSC
        FROM GOSC, POKOJ, POBYT
        WHERE GOSC.ID_POBYTU=POBYT.ID_POBYTU AND
        POBYT.NUMER_POKOJU=POKOJ.NUMER_POKOJU
        AND GOSC.ID_POBYTU = new_name.ID_POBYTU
        GROUP BY GOSC.ID_POBYTU, POKOJ.LICZBA_MIEJSC
        HAVING COUNT(GOSC.ID_GOSCIA)>= POKOJ.LICZBA_MIEJSC
    )
    THEN
        RAISERROR 20002 'Nie mozesz dodac wiecej gosci do tego pobytu'
    END IF
END
MYSQL
DELIMITER $$
```

```
CREATE TRIGGER tr_dodawanie_do_gosc_INSERT
BEFORE INSERT ON GOSC
FOR EACH ROW
BEGIN
    DECLARE v_liczba_obecnnych INT;
    DECLARE v_limit_miejsc INT;
```

Oskar Gregorczyk

```
SELECT COUNT(*) INTO v_liczba_obecných
FROM GOSC
WHERE ID_POBYTU = NEW.ID_POBYTU;

SELECT p.LICZBA_MIEJSC INTO v_limit_miejsc
FROM POKOJ p
JOIN POBYT pb ON p.NUMER_POKOJU = pb.NUMER_POKOJU
WHERE pb.ID_POBYTU = NEW.ID_POBYTU;

IF v_liczba_obecných >= v_limit_miejsc THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Nie mozesz dodac wiecej gosci do tego pobytu';
END IF;
END $$

DELIMITER ;
DELIMITER $$

CREATE TRIGGER tr_dodawanie_do_gosc_UPDATE
BEFORE UPDATE ON GOSC
FOR EACH ROW
BEGIN
    DECLARE v_liczba_obecných INT;
    DECLARE v_limit_miejsc INT;

    IF NEW.ID_POBYTU <> OLD.ID_POBYTU THEN

        SELECT COUNT(*) INTO v_liczba_obecných
        FROM GOSC
        WHERE ID_POBYTU = NEW.ID_POBYTU;

        SELECT p.LICZBA_MIEJSC INTO v_limit_miejsc
        FROM POKOJ p
        JOIN POBYT pb ON p.NUMER_POKOJU = pb.NUMER_POKOJU
        WHERE pb.ID_POBYTU = NEW.ID_POBYTU;

        IF v_liczba_obecných >= v_limit_miejsc THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Nie mozesz dodac wiecej gosci do tego pobytu';
        END IF;
    END IF;
END
```

```
END IF;
END $$
```

DELIMITER ;

Recepcjonista uzupełnia tabelę gości, ten wyzwalacz blokuje potencjalne błędy, gdy omyłkowo pracownik będzie chciał dodać do danego pobytu gościa, który przekraczałby limit, jakim jest liczba łóżek w zarezerwowanym pokoju.

UŻYTKOWNICY

1. Recepcjonista

Hasło: Recepcjonista

Recepcjonista									
Memberships External Logins Table Permissions View Permissions Procedure Permissions Sequenc..									
Table Permissions									
	Name ▲	Owner	Grantors	S	I	D	U	A	R
1	GOSC	DBA	DBA	✓	✓		✓		
2	KLIENT	DBA	DBA	✓	✓		✓		
3	POBYT	DBA	DBA	✓	✓		✓		
4	POKOJ	DBA	DBA	✓					

Recepcjonista							
Memberships External Logins Table Permissions View Permissions Procedure Permiss							
	Name ▲	Owner	Grantors	S	I	D	U
1	StatusPok...	DBA	DBA	✓			

Recepcjonista

Memberships External Logins Table Permissions

	Name ▲	Owner	Execute
1	 KolorOpaski	DBA	✓
2	 StanPokoj...	DBA	✓
3	 Wymelduj...	DBA	✓

```
CREATE USER 'Recepcjonista'@'%' IDENTIFIED BY 'Recepcjonista';
```

```
GRANT SELECT, INSERT, UPDATE ON GOSC TO 'Recepcjonista'@'%';
```

```
GRANT SELECT, INSERT, UPDATE ON KLIENT TO 'Recepcjonista'@'%';
```

```
GRANT SELECT, INSERT, UPDATE ON POBYT TO 'Recepcjonista'@'%';
```

```
GRANT SELECT ON POKOJ TO 'Recepcjonista'@'%';
```


Oskar Gregorczyk

```
GRANT SELECT ON DzisiejszyStatusPokoju TO 'Recepcjonista'@'%';
```

```
GRANT EXECUTE ON FUNCTION KolorOpaski TO 'Recepcjonista'@'%';
```

```
GRANT EXECUTE ON FUNCTION StanPokojuWDniu TO 'Recepcjonista'@'%';
```

```
GRANT EXECUTE ON PROCEDURE WymeldujKlienta TO 'Recepcjonista'@'%';
```

To osoba, która pracuje na recepcji, więc odpowiada za edytowanie tabeli POBYT, KLIENT, a także GOSC. Może również robić kwerendy na tabeli POKOJ, aby na przykład sprawdzić dostępność pokoju. Wykorzystuje widok StatusPokojow. Ma dostęp do funkcji StanPokojuWDniu oraz KolorOpaski, a także do procedury WymeldujKlienta.

2. ManagerSPA

Hasło: ManagerSPA

 ManagerSPA

Memberships	External Logins	Table Permissions	View Permissions	Procedure Permissions	Sequenc..				
Table Permissions									
	Name ▲	Owner	Grantors	S	I	D	U	A	R
1	 GOSC	DBA	DBA	✓					
2	 USLUGA	DBA	DBA	✓	✓	✓	✓		
3	 WYKORZY...	DBA	DBA	✓	✓		✓		

ManagerSPA

Memberships

External Logins

Table Permissions

View Permissions

Procedure Permiss

	Name ▲	Owner	Grantors	S	I	D	U
1	 Popularno...	DBA	DBA	✓			


ManagerSPA

Memberships

External Logins

Table Permissions

	Name ▲	Owner	Execute
1	 StanPokoj...	DBA	✓

```
CREATE USER 'ManagerSPA'@'%' IDENTIFIED BY 'ManagerSPA';
```

```
GRANT SELECT ON GOSC TO 'ManagerSPA'@'%';
```

```
GRANT SELECT, INSERT, UPDATE, DELETE ON USLUGA TO 'ManagerSPA'@'%';
```

```
GRANT SELECT, INSERT, UPDATE ON WYKORZYSTANE_USLUGI TO 'ManagerSPA'@'%';
```

```
GRANT SELECT ON PopularnoscUslug TO 'ManagerSPA'@'%';
```

Bazy Danych – projekt

Oskar Gregorczyk

```
GRANT SELECT ON DzisiejszyStatusPokoju TO 'ManagerSPA'@'%';
```

```
GRANT EXECUTE ON function StanPokojuWDniu TO 'ManagerSPA'@'%';
```

Manager SPA, jest odpowiedzialny za pracę punktu SPA w hotelu. Może edytować i przeglądać tabelę dostępnych usług USLUGA, a także dodawać i edytować rekordy w tabeli WYKORZYSTANE_USLUGI. Manager ma też możliwość przeglądać listę gości hotelowych GOSC. Wykorzystuje widok PopularnoscUslug i StatusPokojuow.

3. Kierownik

Hasło: Kierownik

Kierownik									
Memberships External Logins Table Permissions View Permissions Procedure Permissions Sequenc...									
Table Permissions									
	Name ▲	Owner	Grantors	S	I	D	U	A	R
1	PRACOW...	DBA	DBA	✓	✓	✓	✓		
2	PRODUKT...	DBA	DBA	✓	✓	✓	✓		
3	SPRZATA...	DBA	DBA	✓	✓		✓		
4	ZUZYCIE	DBA	DBA	✓	✓		✓		

Kierownik							
Memberships External Logins Table Permissions View Permissions Procedure Permission							
	Name ▲	Owner	Grantors	S	I	D	U
1	Dzisiejszy S...	DBA	DBA	✓			
2	Efektywno...	DBA	DBA	✓			

Kierownik			
Memberships External Logins Table Permissions View P			
	Name ▲	Owner	Execute
1	RaportSpr...	DBA	✓
2	Sprzatanie...	DBA	✓
3	StanPokoj...	DBA	✓
4	UzupelnijM...	DBA	✓

```
CREATE USER 'Kierownik'@'%' IDENTIFIED BY 'Kierownik';
```

```
GRANT SELECT, INSERT, UPDATE, DELETE ON PRACOWNIK TO 'Kierownik'@'%';
```

```
GRANT SELECT, INSERT, UPDATE, DELETE ON PRODUKTY_Minibarowe TO  
'Kierownik'@'%';
```

Bazy Danych – projekt

Oskar Gregorczyk

```
GRANT SELECT, INSERT, UPDATE ON SPRZATANIE TO 'Kierownik'@'%';
```

```
GRANT SELECT, UPDATE ON ZUZYCIE TO 'Kierownik'@'%';
```

```
GRANT SELECT ON EfektywnoscPracy TO 'Kierownik'@'%';
```

```
GRANT SELECT ON DzisiejszyStatusPokoju TO 'Kierownik'@'%';
```

```
GRANT EXECUTE ON PROCEDURE RaportSprzatan TO 'Kierownik'@'%';
```

```
GRANT EXECUTE ON FUNCTION SprzataniePokoju TO 'Kierownik'@'%';
```

```
GRANT EXECUTE ON FUNCTION StanPokojuWDniu TO 'Kierownik'@'%';
```

```
GRANT EXECUTE ON PROCEDURE UzupełnijMinibar TO 'Kierownik'@'%';
```

Kierownik odpowiedzialny jest za zatrudnianie pracowników, więc ma dostęp do edycji tabeli PRACOWNIK, ustala też zasoby produktów dostępnych w minibarze, czyli dostęp do tabeli PRODUKTY_MINIBAROWE. Wykorzystuje widok StatusPokoju. Ma dostęp do funkcji SprzataniePokoju i StanPokojuWDniu, a także do procedur RaportSprzatan i UzupełnijMinibar.